

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа №5

Выполнила:

Маркозубова Анастасия

К3339

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

1 Задача

- подключить и настроить rabbitMQ/kafka;
- реализовать межсервисное взаимодействие посредством rabbitMQ/kafka.

2 Ход работы

В работе необходимо было выполнить следующие действия:

- подключить брокер сообщений RabbitMQ к проекту;
- настроить exchange и очереди для сервисов;
- реализовать публикацию событий при выполнении бизнес-операций;
- реализовать получение событий другими сервисами;
- проверить работу обмена сообщениями через логи и RabbitMQ Management UI.

RabbitMQ поднимается через Docker Compose. В конфигурации проекта используется образ `rabbitmq:3-management`, который открывает два порта: 5672 для AMQP-подключения сервисов и 15672 для веб-интерфейса управления.

Параметр	Значение
AMQP URL	amqp://localhost:5672
Management UI	http://localhost:15672
Логин / пароль	guest / guest
Exchange	job_board.events
Тип exchange	topic

В каждом микросервисе добавлен файл `src/common/message-bus.ts`. Он отвечает за подключение к RabbitMQ, создание exchange, публикацию событий и подписку сервиса на события других сервисов.

Все сервисы подключаются к общему exchange `job_board.events`. Для каждого сервиса создается собственная очередь с именем вида `<service>.events`. Очереди подписываются на routing key '#', поэтому сервис может получать доменные события, опубликованные другими сервисами.

Сервис	Очередь
auth-service	auth.events
profile-service	profile.events
resume-service	resume.events
vacancy-service	vacancy.events
application-service	application.events
reference-service	reference.events

Общая схема взаимодействия:

1. Пользователь выполняет действие через REST API одного из сервисов.
2. Сервис выполняет бизнес-операцию и публикует событие в RabbitMQ.
3. RabbitMQ маршрутизирует событие через exchange job_board.events.
4. Другие сервисы получают событие из своих очередей и фиксируют получение в логах.

В проекте реализована публикация следующих событий:

Событие	Когда публикуется
auth.user.registered	После регистрации пользователя.
profile.seeker.created	После создания профиля соискателя.
profile.seeker.updated	После обновления профиля соискателя.
profile.company.created	После создания профиля компании.
profile.company.updated	После обновления профиля компании.
resume.created	После создания резюме.
resume.updated	После обновления резюме.
resume.deleted	После удаления резюме.
resume.education.created	После добавления образования в резюме.
resume.experience.created	После добавления опыта работы в резюме.
vacancy.created	После создания вакансии.
vacancy.updated	После обновления вакансии.

<code>vacancy.deleted</code>	После удаления вакансии.
<code>vacancy.application.requested</code>	После отправки отклика на вакансию.
<code>application.created</code>	После создания отклика.
<code>application.status.updated</code>	После изменения статуса отклика.

3 Вывод

В результате выполнения домашней работы в проект сайта поиска работы был добавлен RabbitMQ. Сервисы получили возможность обмениваться доменными событиями через брокер сообщений, не обращаясь друг к другу напрямую. Такой подход снижает связанность микросервисов, упрощает расширение системы и позволяет независимо развивать отдельные компоненты приложения.

Работоспособность решения проверяется через RabbitMQ Management UI и логи сервисов: после выполнения операций в API события публикуются в exchange `job_board.events` и принимаются очередями других сервисов.